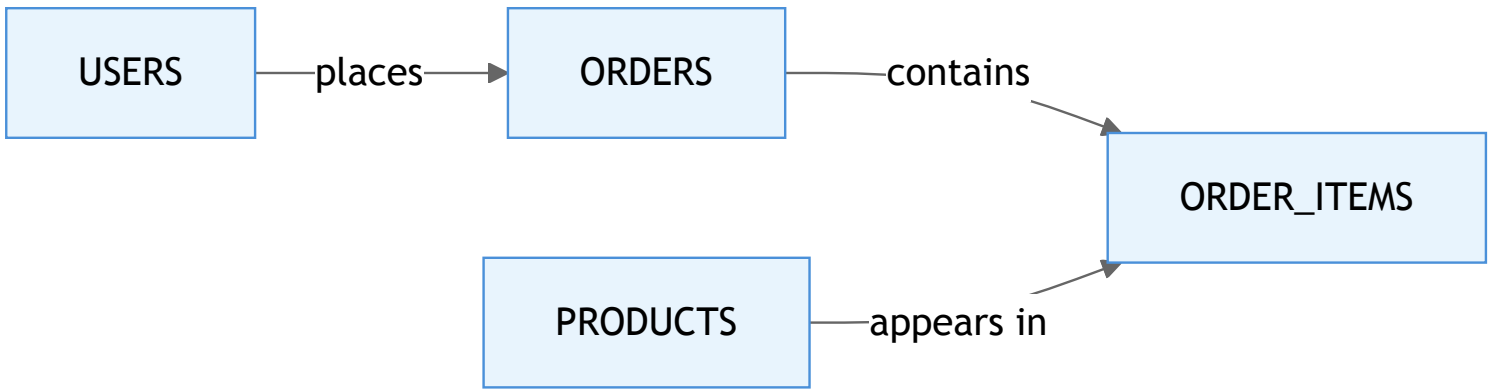
Introduction to SQL

From relational databases to Prisma ORM

What Is a Relational Database?

- Data stored in **tables** (rows and columns)
- Tables linked through **relationships**
- Enforced structure via a **schema**
- Queried using **SQL** (Structured Query Language)



Normalisation

Design technique to **reduce redundancy** and **improve data integrity**

Normal Form	Rule
1NF	Atomic values, no repeating groups
2NF	1NF + no partial dependencies
3NF	2NF + no transitive dependencies

Benefits: consistency, simpler updates, less wasted storage

First Normal Form (1NF)

Each cell holds **one value**. Every row is **unique**.

Before 1NF:

```
Student | Courses
————————┼———————————————
John    | Math, Science, Art
```

After 1NF:

```
Student | Course
————————┼————————
John    | Math
John    | Science
John    | Art
```

Second Normal Form (2NF)

1NF + every non-key column depends on the **entire** primary key

Before 2NF (composite key: StudentID, CourseID):

```
StudentID | CourseID | StudentName | CourseName
──────────┼──────────┼─────────────┼────────────
1         | 101      | John        | Math
```

After 2NF:

```
Students:   StudentID | StudentName
Courses:    CourseID  | CourseName
Enrolment:  StudentID | CourseID
```

Third Normal Form (3NF)

2NF + no **transitive dependencies** between non-key columns

Before 3NF:

```
StudentID | Name | Department | DeptHead
──────────┼──────┼────────────┼──────────
1         | John | CS         | Dr. Smith
```

After 3NF:

```
Students:    StudentID | Name | DeptID
Departments: DeptID    | Department | DeptHead
```

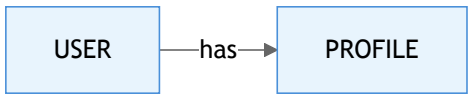
Relationships

How tables connect to each other through **keys**

- **One-to-One** – User → Profile
- **One-to-Many** – User → Posts
- **Many-to-Many** – Posts ↔ Tags (via junction table)

One-to-One Relationship

Each record in Table A relates to **exactly one** record in Table B



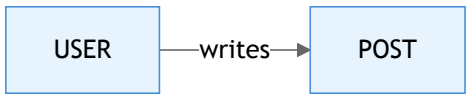
Example: A user has one profile, a profile belongs to one user

Implementation: Foreign key with `UNIQUE` constraint

```
CREATE TABLE profiles (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL UNIQUE REFERENCES users(id),
    bio TEXT
);
```

One-to-Many Relationship

One record in Table A relates to **multiple** records in Table B



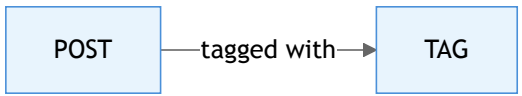
Example: A user writes many posts, each post belongs to one user

Implementation: Foreign key on the "many" side

```
CREATE TABLE posts (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL REFERENCES users(id),
    title VARCHAR(255)
);
```

Many-to-Many Relationship

Records in Table A relate to **multiple** records in Table B and vice versa



Example: Posts have many tags, tags belong to many posts

Implementation: **Junction table** (join table) with two foreign keys

```
CREATE TABLE post_tags (
    post_id INT NOT NULL REFERENCES posts(id),
    tag_id INT NOT NULL REFERENCES tags(id),
    PRIMARY KEY (post_id, tag_id)
);
```

Keys

- **Primary Key (PK)** – Uniquely identifies each row
- **Foreign Key (FK)** – References a PK in another table
- **Composite Key** – PK made of multiple columns

```
CREATE TABLE posts (
    id SERIAL PRIMARY KEY,
    title VARCHAR(255) NOT NULL,
    user_id INT NOT NULL REFERENCES users(id)
);
```

SQL – Structured Query Language

Three sub-languages: **DDL**, **DML**, **DQL**

DDL – Data Definition Language

Define and modify the **structure** of your database

```sql
-- Create a table
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT NOW()
);

-- Add a column
ALTER TABLE users ADD COLUMN username VARCHAR(50);

-- Remove a table
DROP TABLE users;
```

Key commands: `CREATE`, `ALTER`, `DROP`, `TRUNCATE`

DML – Data Manipulation Language

Insert, **update**, and **delete** data

```sql
-- Insert a row
INSERT INTO users (email, username)
VALUES ('alice@example.com', 'alice');

-- Update existing rows
UPDATE users SET username = 'alice_dev'
WHERE email = 'alice@example.com';

-- Delete specific rows
DELETE FROM users WHERE id = 42;
```

Always use a `WHERE` clause with `UPDATE` and `DELETE`!

DQL – Data Query Language

Read data with `SELECT`

```sql
-- Filter rows
SELECT * FROM users WHERE username LIKE 'alice%';

-- Join tables
SELECT u.username, p.title
FROM users u
JOIN posts p ON u.id = p.user_id;

-- Aggregate
SELECT COUNT(*) AS total, AVG(age) AS avg_age
FROM users;
```

Key clauses: `WHERE` , `JOIN` , `GROUP BY` , `ORDER BY` , `LIMIT`

JOINs Overview

Combine rows from two or more tables based on a matching condition

INNER JOIN
Matching only

LEFT JOIN
All left + match

RIGHT JOIN
All right + match

FULL OUTER
All from both

Four types, each with different behavior when rows don’t match

INNER JOIN – Where Data Overlaps

Returns only rows that have **matching values in both tables**

Only the **intersection** is included in the result

```
SELECT customers.name, orders.order_date
FROM customers
INNER JOIN orders
ON customers.id = orders.customer_id;
```

Result:

- Only customers who have placed at least one order appear
- If a customer never made a purchase, they won't show up
- If an order has no matching customer, it won't show up

Use case: "Show me customers with orders"

LEFT JOIN – Keep Everything from the Left Table

Returns **all rows from the left table** plus matching rows from the right table

Entire **left table** included, right table only where matched

```
SELECT customers.name, orders.order_date
FROM customers
LEFT JOIN orders
ON customers.id = orders.customer_id;
```

Result:

- You see all customers, including those who never placed an order
- Their order_date will be NULL
- Great for finding inactive or new users

Use case: "Show me all customers and their orders (if any)"

RIGHT JOIN – The Mirror of LEFT JOIN

Returns **all rows from the right table** plus matching rows from the left table

Entire **right table** included, left table only where matched

```
SELECT customers.name, orders.order_date
FROM customers
RIGHT JOIN orders
ON customers.id = orders.customer_id;
```

Result:

- You get all orders, even orphaned ones
- If an order was made by a deleted customer, the name will be NULL
- Helpful for finding data inconsistencies

Use case: "Show me all orders and their customers (if any)"

FULL OUTER JOIN – Bring Everything Together

Returns **all rows from both tables**, merging matches and filling unmatched fields with NULL

Everything from both tables is included

```
SELECT customers.name, orders.order_date
FROM customers
FULL OUTER JOIN orders
ON customers.id = orders.customer_id;
```

Result:

- You see all customers and all orders
- Matched pairs are joined normally
- Unmatched customers have NULL in order fields
- Orphaned orders have NULL in customer fields
- Perfect for finding gaps and inconsistencies

Use case: "Show me the complete picture – all customers and all orders"

Choosing the Right JOIN

Situation	Use
Both sides must have matches	INNER JOIN
Keep all left-side rows	LEFT JOIN
Keep all right-side rows	RIGHT JOIN
Keep everything from both	FULL OUTER JOIN
Find unmatched rows	LEFT or RIGHT with WHERE NULL filter

Most common: **LEFT JOIN** (find all X and their related Y, if any)

What Is an ORM?

Object-Relational Mapping – a bridge between code objects and database tables

- Write queries in your **programming language**
- Get **type-safe** database operations
- Automatic **migrations** and schema management
- **Database-agnostic** – swap providers without rewriting queries

Popular ORMs: **Prisma**, TypeORM, Sequelize (Node.js), Hibernate (Java)

Migrations

Version control for your database schema

- Track every schema change as a **timestamped file**
- Apply changes **consistently** across dev, staging, production
- **Rollback** capability when things go wrong
- Team members get the same schema via `migrate deploy`

```
migrations/
  20260301120000_create_users/
    migration.sql
  20260302090000_add_posts/
    migration.sql
```

Prisma – Modern TypeScript ORM

Define your schema, generate a type-safe client

```
model User {
  id        Int      @id @default(autoincrement())
  email     String   @unique
  username  String?
  posts     Post[]
  createdAt DateTime @default(now())
}

model Post {
  id     Int    @id @default(autoincrement())
  title  String
  user   User   @relation(fields: [userId], references: [id])
  userId Int
}
```

Prisma in Action

```
// Create a user with a post
const user = await prisma.user.create({
  data: {
    email: "alice@example.com",
    posts: {
      create: { title: "Hello World" },
    },
  },
});

// Query with relations
const users = await prisma.user.findMany({
  include: { posts: true },
});
```

Features: auto-complete, type safety, Prisma Studio GUI

Optimisation

Strategies to keep your database **fast**

Strategy	What It Does
Indexing	Speed up lookups on frequently queried columns
Query analysis	Use `EXPLAIN ANALYZE` to find slow queries
Avoid N+1	Fetch related data in one query, not N extra ones
Connection pooling	Reuse connections instead of opening new ones
Caching	Store hot data in memory (Redis)

Security

Protecting your data from threats

- **SQL Injection** – Never concatenate user input into queries

```
-- DANGEROUS
SELECT * FROM users WHERE name = '' + userInput + '';

-- SAFE – parameterized query
SELECT * FROM users WHERE name = $1;
```

- **Least Privilege** – App accounts get only the permissions they need
- **Encryption** – Encrypt sensitive data at rest and in transit (TLS)
- **Backups** – Regular, tested, automated backups

NoSQL

When relational isn't the right fit

Type	Example	Use Case
Document	MongoDB	Flexible schemas, nested data
Key-Value	Redis	Caching, sessions
Column-Family	Cassandra	High-write, time-series
Graph	Neo4j	Relationship-heavy queries

Trade-offs: flexibility vs consistency, scale vs ACID guarantees

Summary

Topic	Key Takeaway
Normalisation	1NF→2NF→3NF
Relationships	1:1, 1:many, many:many
SQL	DDL/DML/DQL commands
ORM	Type-safe access
Migrations	Versioned schema
Optimisation	Indexes, N+1, pooling
Security	Parameterized queries

Next Up: Hands-on with Prisma

Questions?